1 Implementation Guide

This document describes the implementation and connection of a production line to the product group "Dynamic Manufacturing Control" (DMC). The document also explains the basic terms and the required steps to configure DMC.

Further applicable documents

The following documents contain basic technical information on DMC or supplement this document:

- Hardware and software recommendations
- DMC SDK (included in DMC delivery)
    - DMC API documentation
    - Sample ImplementationGuide

Overview

In this document, first the example of a reference process shows how a reference line is connected to DMC.

Next, a data model of the physical line, the so-called factory model, is created. It digitally represents the components of the physical world, e.g. workstations and peripherals, in the Dynamic MES Weaver (DMW) as software components. Based on the factory model, the physical production process is described. The data model specifies the production processes on the line. This data model is referred to as manufacturing instruction in the entire documentation.

The document then deals with the generation of workpieces. The production order (that includes information on the product variant), the manufacturing instruction and the factory model are the basis for the workpieces. Each workpiece digitally represents one physical item. The workpiece is a kind of data container filled with all relevant information to produce the item. During the production in the DMW, the system records and displays all status information referring to the production process.

The document then deals with further aspects of the integration in HYDRA, e.g. Machine Data Collection MDE and Material and Production Logistics MPL. At the end, the document deals with the deployment of the DMC instances.

2 Reference process

The following paragraph shows how to connect a production line. A reference process is used as example. The reference process shows the production of an instrument panel on a line with five workstations and a rework station. The process is fictional. It has been designed to show how the system works.

Reference line

The illustration below Figure 1 Reference line) depicts the reference line.

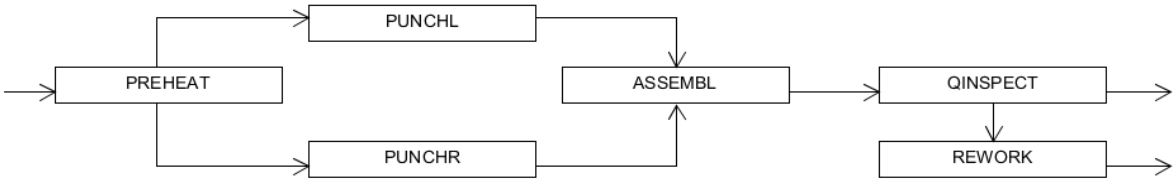

Figure 1 Reference line

The structure and stations of the line are described in the following. The station PREHEAT is at the beginning of the line. Depending on the product variant, the item is directed to the station PUNCHL or PUNCHR. The process goes on in the station ASSEMBL. The item then proceeds to the station QINSPECT for quality inspection. If necessary, the item is passed to the REWORK station.

A connected scanner at the PREHEAT station identifies materials. A heating unit can be activated manually to join parts. The station then passes the item on to one of the two subsequent workstations via a switch.

The stations PUNCHL and PUNCHR also have a scanner. A punching process is manually performed. Once finished, the item is forwarded to the next workstation.

The station ASSEMBL screws input material to the item. The station uses a scanner and a screwdriver. Once the screwing process is finished, the item is directed to the next station.

The QINSPECT station inspects the quality. The station uses a scanner and a label printer. Depending on the result of the inspection, the item is directed off the line or redirected to the rework station.

The REWORK station uses a scanner and a screwdriver. All stations of the line have a terminal.

Production variants

An option code that is made up of two parts defines the variants of the reference process. For each variant of the reference process, there are two possible options. As a result, there are four variants (see Table 1).

Table 1 Option codes in the reference process

		Part 2	
		4Y0	**4Y1**
Part 1	**L0L**	L0L-4Y0	L0L-4Y1
	L0R	L0R-4Y0	L0R-4Y1

List of parts

For each variant, different parts must be installed. Table 2 List of parts) shows an overview.

Table 2 List of parts

Option code	Panel	Parts to be installed
L0L-4Y0	100-100	100-200, 600-100, 600-300
L0L-4Y1	100-100	100-200, 600-100, 600-200, 600-300
L0R-4Y0	100-101	100-201, 600-100, 600-301
L0R-4Y1	100-101	100-201, 600-100, 600-200, 600-301

Process and work steps

According to the structure of the reference line, the production is divided into 6 process steps. Each workstation of the line corresponds to one process step including several work steps. They are listed in Table 3 Process and work steps).

Table 3 Process and work steps

Process step	Work step	Comment
0010		**Heat**
	0010.0010	Scan panel 100-100
	0010.0011	Scan panel 100-101
	0010.0020	Scan part 100-200
	0010.0021	Scan part 100-201
	0010.0030	Heat

	0010.0040	Direct to the left
	0010.0041	Direct to the right
0040		**Punch left**
	0040.0010	Scan panel 100-100
	0040.0020	Punch
0041		**Punch right**
	0041.0011	Scan panel 100-101
	0041.0020	Punch
0060		**Assembly**
	0060.0010	Scan panel 100-100
	0060.0011	Scan panel 100-101
	0060.0020	Scan part 600-100
	0060.0030	Screw part 600-100
	0060.0040	Scan part 600-200
	0060.0050	Screw part 600-200
	0060.0060	Scan part 600-300
	0060.0061	Scan part 600-301
	0060.0070	Screw part 600-300 / 600-301
0070		**Quality control**
	0070.0010	Scan panel 100-100
	0070.0011	Scan panel 100-101
	0070.0020	Poka Yoke
	0070.0030	Quality control
	0070.0040	Print label
0080		**Rework**
	0080.0010	Scan panel 100-100
	0080.0011	Scan part 100-101
	0080.0020	Poka Yoke

	0080.0030	Rework

Process description

The valid transitions between the individual process steps are shown in Figure 2 Process flow). If an item is identified as scrap, the process is stopped. Otherwise, processing is continued. The option code specifies the transition from process step 0010 to the subsequent step (L0L: go on with step 0040; L0R go on with step 0041). If the quality control (process step 0070) identifies the item as rework, the item is forwarded to the rework process step (0080).

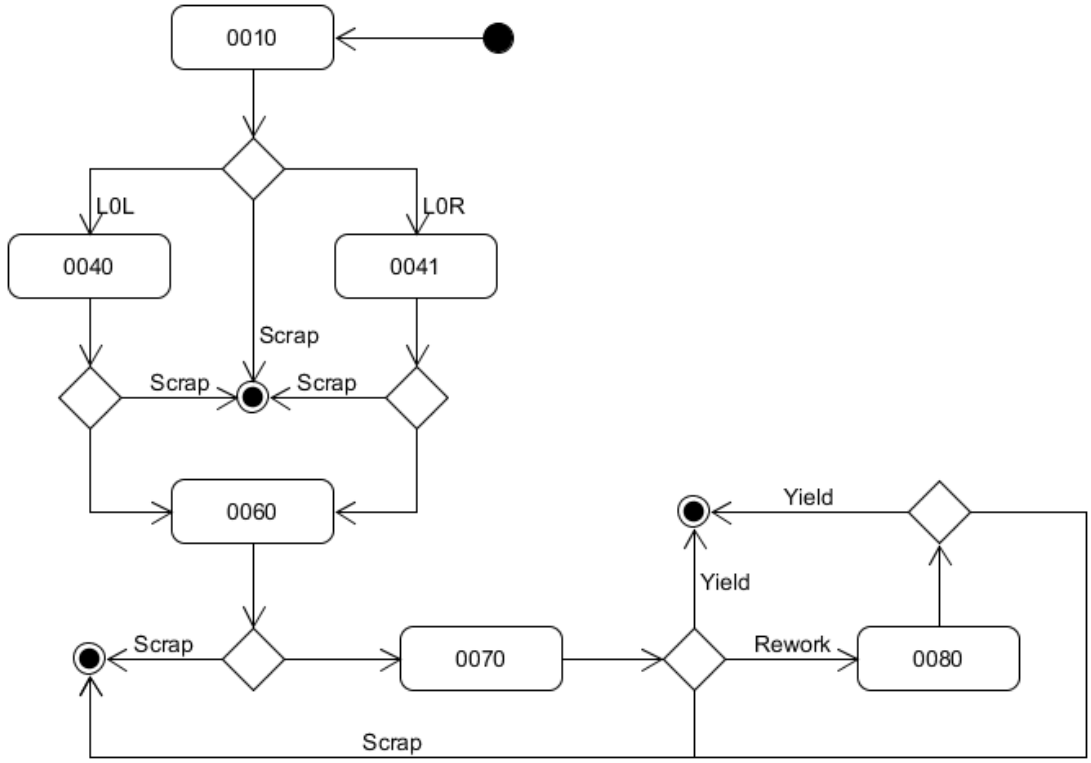

Figure 2 Process flow

The flow of process step 0010 is shown in Figure 3 Process step 0010). Depending on the option code, work step 0010.0010 or 0010.0011 is performed at the beginning. The workstation scans the panel and afterwards the parts to be installed. After successful scanning, the item is further processed in step 0010.0030. When the item leaves the station, the option code defines if step 0010.0040 or 0010.0041 is performed. If an item is identified as scrap in one of the work steps, the process is stopped.

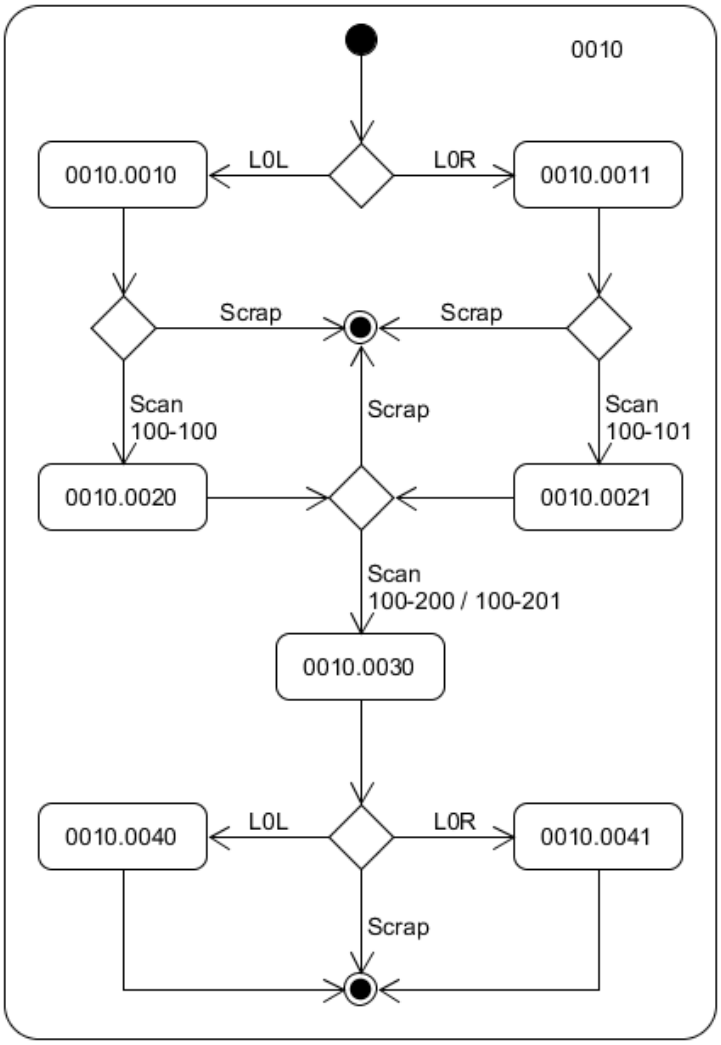

Figure 3 Process step 0010

Process steps 0040 and 0041 have a corresponding structure. Figure 4 Process step 0040) shows process step 0040 as an example. The station scans the panel and performs work step 0040.0020. If an item is identified as scrap, the process is interrupted.

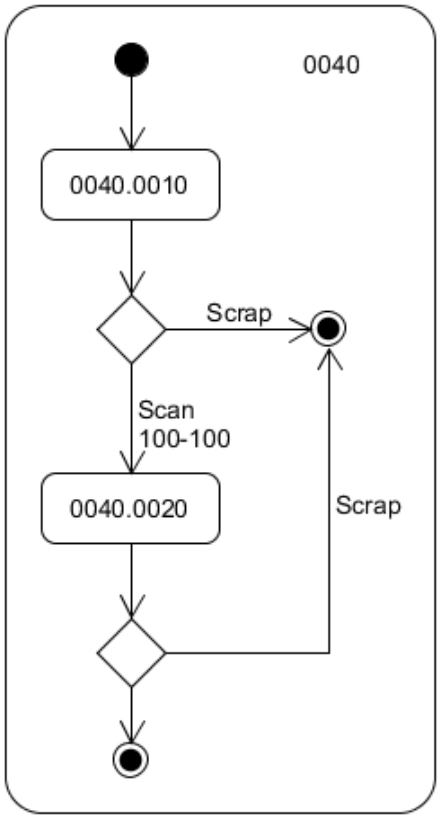

Figure 4 Process step 0040

Process step 0060 is shown in Figure 5 Process step 0060). Depending on the option code, the workstation scans the panel in work step 0060.0010 or 0060.0011. The workstation scans the part 600-100 in step 0060.0020 and if identified, the part is screwed on in step 0060.0030.

Depending on the option code, part 600-200 is scanned and screwed. This is controlled by the option 4Y1 and realized in work steps 0060.0040 and 0060.0050. Once the process is finished or if option 4Y0 is valid, an additional part is scanned and screwed.

The corresponding option code controls which part is scanned and screwed next. The option L0L scans part 600-300 in step 0060.0060 and option L0R scans part 600-301 in step 0060.0061. With both options, the screws are tightened and the process step is completed.

Processing is stopped, if an item is identified as scrap in one of the work steps.

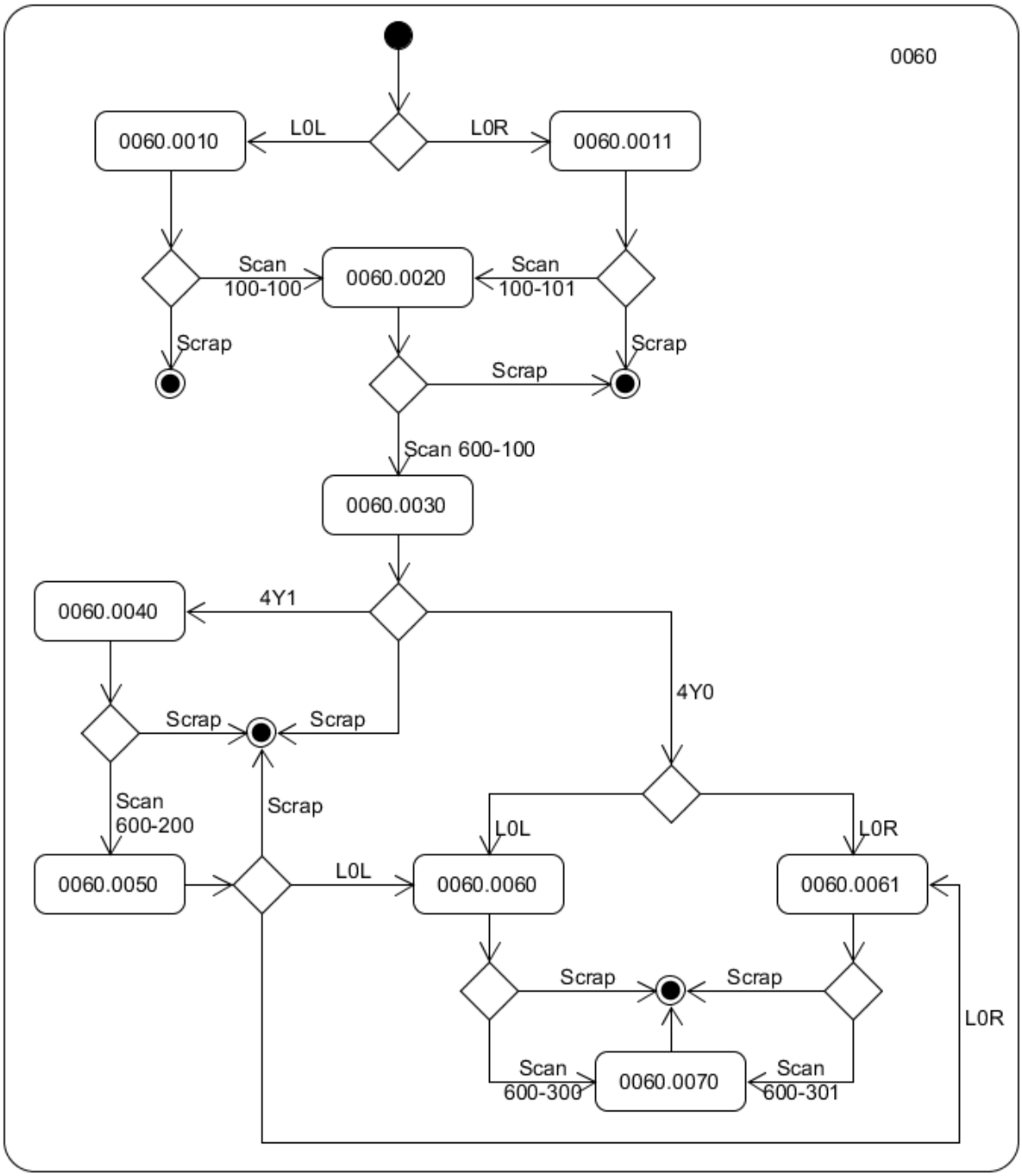

Figure 5 Process step 0060

Once the assembly is finished, the quality control is performed. This is shown in Figure 6 Process step 0070). Depending on the option code, work step 0070.0010 or 0070.0011 identifies the panel. The condition of the item is evaluated in the next work step via poka-yoke. If the item is not identified as scrap, it is passed to the quality control. If the item passes the inspection (yield), a label is printed. If the item is identified as scrap, the process is stopped.

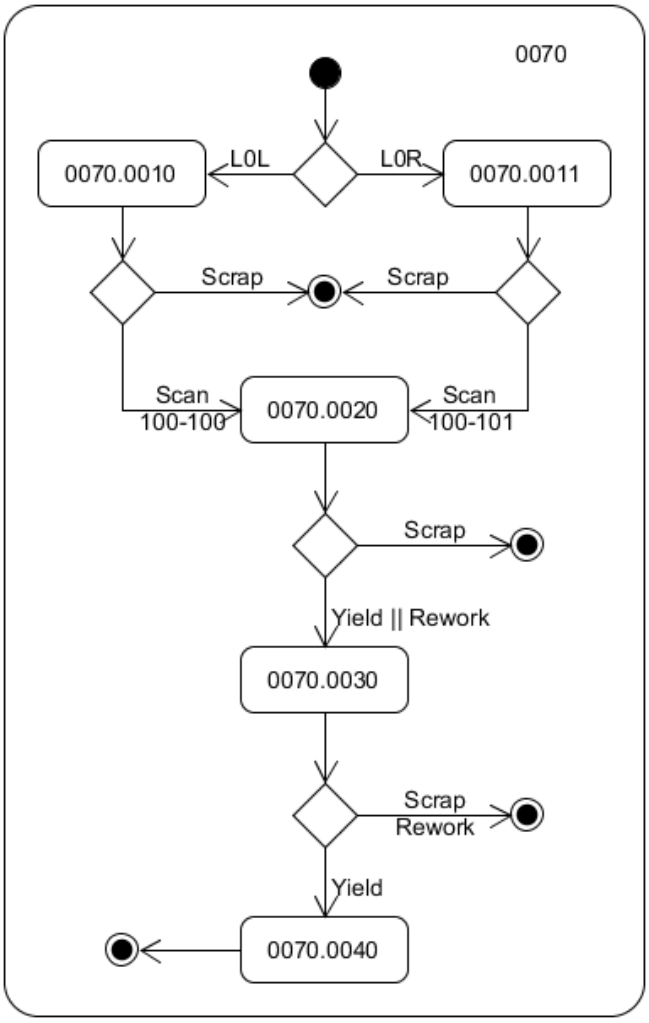

Figure 6 Process step 0070

If the item is classified as rework, process step 0080 is performed. This step is shown in Figure 7 Process step 0080). Depending on the option code, the workstation scans and identifies the panel also in this work step.

The condition of the item is evaluated in the next work step via poka-yoke. The rework step is started if the item is not identified as scrap.

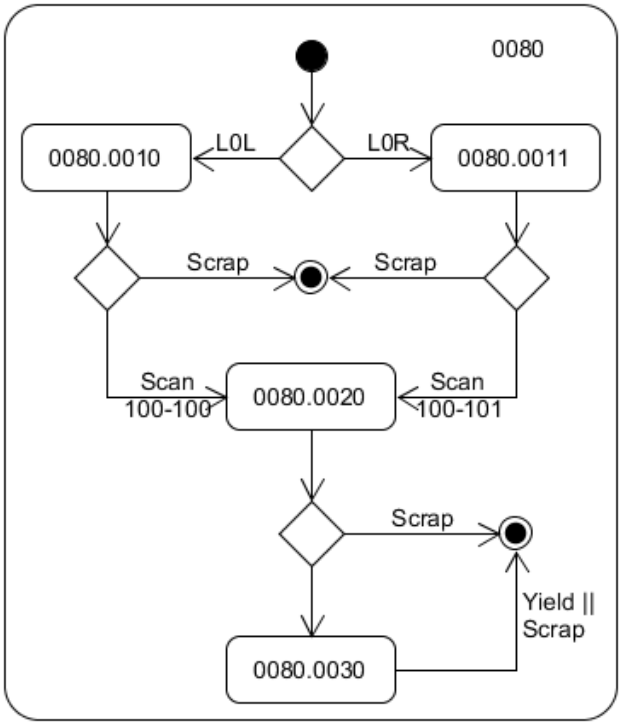

Figure 7 Process step 0080

If the condition of the item can be rectified during the rework, the item is identified as yield. If the rework process is not successful, the item is identified as scrap. In both cases, the process step and the production process are then completed.

3 Data modeling of the line

This chapter shows how to create a data model of the physical line. The data model digitally represents the components of the physical world, e.g. workstations and peripherals. In the DMW, the physical components are represented as software components. Based on this data model, the next step can digitally show the production process of the physical world.

Data model

The data model of the line is called factory model. The factory model defines the software components that represent the physical components of the line. The factory model also defines the configuration of these components. The factory model has a hierarchical structure. It is shown in the illustration below, Figure 8 Factory model).

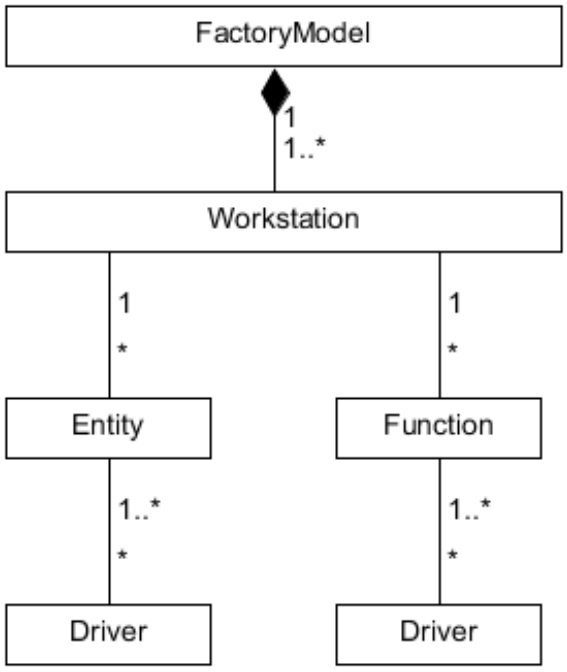

Figure 8 Factory model

The workstations of the line are the highest level. Each workstation includes components that connect peripherals (entity) and components that offer special functions, e.g. the identification of the machine status (function). If necessary, drivers are additionally assigned to the components that ensure communication with the PLC (programmable logic controller).

In the reference process, the factory model consists of 6 workstations. The entity "scanner" is here part of the station PREHEAT. The scanner uses a serial driver to communicate with the hardware.

As a functional component, the PREHEAT station is equipped with a MaterialManager component which, among other things, is responsible for the validation of the installed material.

File format

The factory model must be created as XML file. The basic structure of this file corresponds to the DMC configuration file format. For each software component of the factory model, a ComponentConfiguration entry is included in the file. The names of the components assigned to a station are stored in the children attribute and must inherit from the *CapableComponent* class.

```
<CollectorConfiguration Version="1.0">
  <ComponentConfiguration Name="ComponentX" Class="…" Children="…">
    <Parameter Name="Param1" Value="…" />
     …
    <Parameter Name="ParamN" Value="…" />
  </ComponentConfiguration>
   …
</CollectorConfiguration>
```

As an example, the file *fm_reference.xml* shows the structure of the factory model of the reference process. The following section describes selected components used in the reference process.

Model components

Selected components of the reference process are presented here. Depending on the line that must be connected, the use of other components is possible. You can find an overview of the components that digitally represent HYDRA functions (MDE, MPL, PDV,...) in paragraph 0.

If necessary, you must create new and specialized components (see also paragraph *Use of the data model).*

Workstation: In general, you must configure a workstation component for each station of the line. The component name must correspond to the workplace ID of the station in HYDRA. You must configure all functions and peripherals assigned to the station as children.

TerminalAdapter: If you connect a terminal to a station, you must configure the TerminalAdapter component and assign it to the corresponding workstation component. During runtime, a GUI component must connect to the terminal adapter. Obviously, it is also possible to connect a terminal via another component. Find details on this subject in paragraph 0**Error! Reference source not found.**.

Scanner

SerialPortDriver

ConfigurableTrigger

ConfigurableToggle

ConfigurableOutput

Bridge: You must configure the component *Bridge* if you retrieve data from a machine via MPDV PCC-drivers (opcmpdv.dll – OPC-DA - Delphi,opcua_mpdv.dll, OPC-UA - C++).

For the PCC drivers, you must make a corresponding INI configuration (opcmpdv.ini or opcua_mpdv.ini). You will find further details in the PCC OPC instruction.
(**TODO LINK**).
The document DMC_Bridge.docx describes the configuration settings of this component.
For further information, please refer to the API documentation.

```
<ComponentConfigurationName="OpcDriver"
             Class="mpdv.MachineDataCollector.PccDriverBridge.Bridge">
    <Parameter Name="PccDrivers_DllPath" Value="Native\x86\OPCMPDV\OPCMPDV.dll" />
    <Parameter Name="PccDrivers_IniPath" Value="Native\x86\OPCMPDV\OPCMPDV.ini" />
    <Parameter Name="PccDrivers_SaveBufferedChannels" Value="N" />
    <Parameter Name="PccDrivers_DelphiDriverWrapper"
               Value="Native\x86\DelphiDriverBridge\LegacyDriverWrapper.dll" />
    <Parameter Name="PccDrivers_RequiresMessagePump" Value="True" />
    <Parameter Name="TypeMapping_IntegerParameters"
               Value="O:OD15,O:OD16,M:MSTAT@PREHEAT" />
    <Parameter Name="TypeMapping_BooleanParameters"
               Value="I:ID1,I:ID2,I:ID3,I:ID4,I:ID5,I:ID6,I:I001,
                      I:I002,I:I003,I:I004, I:I005" />
    <Parameter Name="TypeMapping_StringParameters" Value="OPCMPDV_CLIENT" />
</ComponentConfiguration>
```

You must define the bridge in the custom component as child attribute in order to use the bridge. The custom component must inherit from the class *BaseEntityWithDriver* (see DMC API documentation). Only then, the custom component receives events from the bridge via the function *GotDriverValue*.

Creation of components

An important feature of DMC is that it can be extended by custom components. You can therefore perfectly adapt the application to your own processes and requirements. The tutorial *DMC_CreatePlugin* describes the development of a DMC plug-in step by step. To start with, we recommend the document *MBL_DMC_ComponentSystem* as it offers an overview of the component system. You will find further details in the samples included in the DMC SDK delivery (especially *Component* and *CapableComponent*).

Use of the data model

During runtime, the DMW instantiates the components of the factory model when using the DMW configuration template. In order to integrate the model of the reference line (fm_reference.xml), the DMW configuration can be as follows (config_server.xml):

```
<CollectorConfiguration Template="DMW"
                        FactoryModel=" fm_reference.xml"
                        TerminalId="12"/>
```

Please note that a relative path for the factory model is resolved relative to the path of the configuration file. You must also ensure that component names are unique names also beyond configuration and factory model. The configuration of factory model components can be overwritten in the configuration file.

You must create a configuration for each Dynamic Line Panel (DLP). As an example, the configuration of the DLP of the PREHEAT station is listed in the following:

```
<CollectorConfiguration Template="DLP" Name="Pre Heating" TerminalId="121">
  <ComponentConfiguration Name="CommunicationComponent">
    <Parameter Name="Protocol" Value="HTTP" />
    <Parameter Name="Address" Value="http://127.0.0.1:3270" />
    <Parameter Name="Serialization" Value="Json" />
  </ComponentConfiguration>
  <ComponentConfiguration Name="RegistryClient">
    <Parameter Name="ComponentsToRegister" Value="PreHeatingGui" />
  </ComponentConfiguration>
  <ComponentConfiguration Name="PreHeatingPresenter"
             Class="mpdv.MachineDataCollector.Dmc.Gui.Presenters.HomePresenter" />
  <ComponentConfiguration Name="PreHeatingGui"
             Class="mpdv.MachineDataCollector.Dmc.Entities.TerminalAdapter"
             Children="PreHeatingPresenter" />
</CollectorConfiguration>
```

The DLP configurations of the reference process are structured according to this configuration (config_preheating.xml, config_punchingleft.xml, config_punchingright.xml, config_assembling.xml, config_qualityinspection.xml, config_rework.xml).

We recommend to use the template *DLP* for the DLP configuration. This template configures the components needed for the DMW communication. You must adjust the following settings in the configuration:

The component *CommunicationComponent* is used for the basic DMW communication. Set the parameter *Address* according to the DMW configuration. In the example, the parameter is set to the value http://127.0.0.1:3270. It is assumed that DLP and DMW are carried out in the same host and http is used for communication.

Configure the parameter *ComponentsToRegister* for the component *RegistryClient*. It is a comma-separated list of all component names that must be registered in the DMW, i.e. all components that directly interact with the DMW. In the example above, it is the component *PreHeatingGui* which is stored as child of the PREHEAT station in the factory model.

The component *PreHeatingGui* is configured according to the factory model. Finally, the component *PreHeatingPresenter* integrating the GUI component is configured. During runtime, this component links to the component *PreHeatingGui.*

In general, you must configure all components of the factory model in the DLP configuration that must be instantiated in the DLP.

4 Data modeling of the production process

The objective of this chapter is to create a data model that describes the manufacturing process using the components of the factory model. This data model is called manufacturing instruction. The manufacturing instructions are intended to cover all variants of the production line. During the instantiation, a digital workpiece is then generated from the manufacturing instructions for each item to be produced. In our context, the workpiece is a digital workpiece and not a physical item. This digital workpiece can finally be executed by the DMW.

Data model

The data model of the manufacturing process is called manufacturing instruction. The manufacturing instruction defines the configuration of the software components of the factory model depending on the variant to be manufactured, the manufacturing step and, if necessary, the state of the production. The manufacturing instruction is basically structured as a list of production steps. The individual steps are subdivided in substeps. For the data modeling of a step, the production steps should correspond to the process steps, i.e. to the production tasks at the station in question. The substeps should then be modeled as individual work steps at the corresponding station.

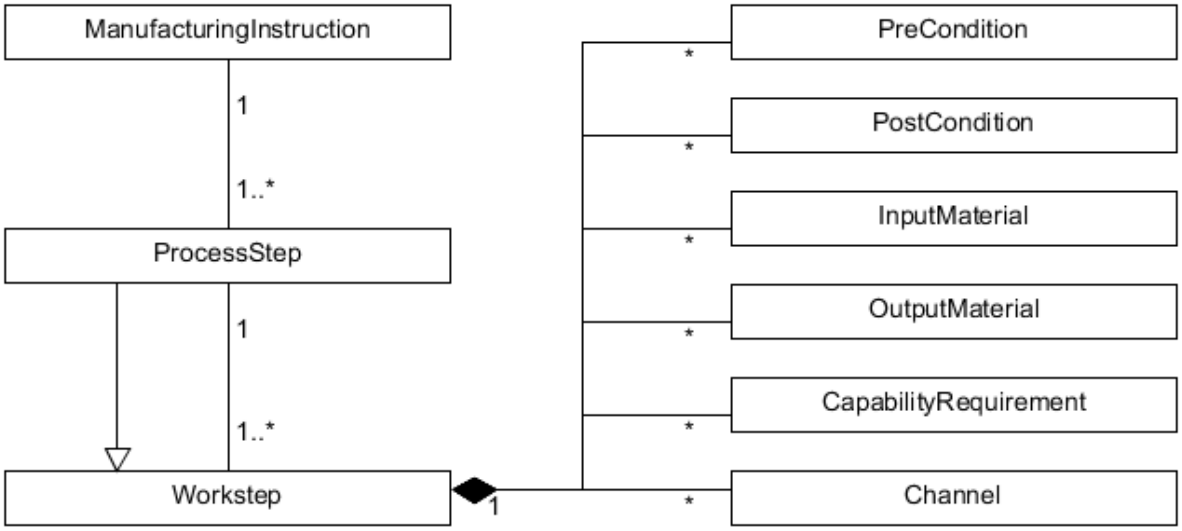

Figure 9 Manufacturing Instruction

Manufacturing Instructions (see Figure 9) include a unique ID and a list of process steps. The process steps include substeps that are the work steps. Process steps and work steps have a unique ID in the manufacturing instruction. From a technical point of view, a process step is identical to a work step and has identical properties.

A work step includes preconditions and postconditions. All preconditions must be fulfilled, before a work step can be performed. A work step is completed once all postconditions are met or if the work step is stopped. Based on these conditions, you can define requirements to interlock the process or you can digitally model the control of variants. The conditions are scripts that represent logical expressions. Please refer to the document *MBL_DMC_Scripting* for further details.

The input and output material is defined for each work step. The input material generally refers to the parts installed during the production process in a specified step. The output material is of informational nature and is not relevant to the production of the item.

In addition, the manufacturing instruction defines for each work step the capability requirements necessary for the production. These requirements specify the peripherals or functions the workstation must provide. In general, the requirements also specify how the station must be configured for the corresponding work step. For example, the torque of a screwdriver might be specified here.

Finally, the manufacturing instruction also defines the communication channels for the process and work steps that are used between the different components of a workstation. For example, you can specify that process data of the screwdriver are transferred to the GUI for live monitoring.

API modeling

In addition to the manufacturing instruction as XML file, we urgently recommend to use the C#-API modeling. Data modeling is much easier using this method and you can thus easily generate an XML file.

The API is based on two main parts in order to create manufacturing instructions:

- Functions (helpers) to model the instructions.
- An abstract component (BaseInstructionCreator) used in combination with a configuration file to generate and persist the modeled instructions during execution of the program.

We use the component to create manufacturing instructions for the reference process (ReferenceInstructionCreator.cs) as an example to show the use of this API.

For this component, you can use e.g. config_instructions.xml:

```
<CollectorConfiguration Version="1.0" Template="Default">
  <ComponentConfiguration Name="ManufacturingInstructionRepository"
      Class="mpdv.MachineDataCollector.Dmc.Generator.ManufacturingInstructionReposit
ory" />
  <ComponentConfiguration Name="ReferenceInstructionCreator"
      Class="mpdv.MachineDataCollector.Dmc.Generator.ReferenceInstructionCreator">
    <Parameter Name="FactoryModel" Value="demo\fm_reference.xml" />
  </ComponentConfiguration>
</CollectorConfiguration>
```

On start of this configuration, the component *ReferenceInstructionCreator* creates the manufacturing instruction (mi_reference.zip) and finishes the program.

The component *ReferenceInstructionCreator* derives from the BaseInstructionCreator. In the configuration, you must indicate the inherited parameter *FactoryModel.* This parameter defines the factory model for which you want to create the manufacturing instructions.

As child class of BaseInstructionCreator, you must only implement the method *CreateManufacturingInstructions.* In this method, you can model the manufacturing process using the API helpers (class *ManufacturingInstructionHelper*) and finally create a manufacturing instruction. The following table offers an overview of the helper functions provided. Please refer to the DMC API documentation for further details on these functions.

Function	**Explanation**
CreateInstructionBuilder	Creates a builder instance for the data modeling of the production process.
CreateProcessStep	Creates a process step.
AddProcessStep	Adds a process step to the builder.
AssignTo	Assigns a workstation to a process step for the processing.
Creatework step	Creates a work step.
AddWorkstep	Adds a work step to a process step.
AddPreCondition	Adds a precondition to a process or work step. If all preconditions are met, the processing of the step can start.
AddStepSuccessPreCondition	Adds a precondition to a process or work step which checks if a specified step has already been completed successfully.
AddOptioncodePreCondition	Adds a precondition to a process or work step which defines the option code for performing this step.
AddPostCondition	Adds a postcondition to a process or work step. If all conditions are met, the step is completed.
AddChannel	Models a communication channel between the output of a capability and the input of another capability.

AddAssesmentRouting	Models channels for process steps for the transfer of events from components with WorkpieceAssessment-Capability to the corresponding stations.
AddMde	Models channels for process steps for the transfer of events between components that offer MDE functionality.
AddInputMaterial	Adds input material to a work step.
AddOutputMaterial	Adds output material to a work step.
AddRequirement	Adds a capability requirement to a work step.
Build	This function creates the manufacturing instruction to complete the modeling.

Use of the data model

All DMC instances involved in the manufacturing process must be able to access the created manufacturing instructions. If you use the configuration templates DMW, DLP and DWG, you must store the manufacturing instructions in the folder *instructions* in the shared data directory. You can change the storage location for the component *ManufacturingInstructionRepository* following the instructions in the DMC API documentation.

It is possible to store several manufacturing instructions for different production processes in one DMC. It is then possible to change from one process version to another version without standstills in the line.

It is then also possible to switch in a flexible line dynamically between the production of different products.

In this case, you must ensure that the stored manufacturing instructions match the factory model of the corresponding line. If the manufacturing instructions were created for different factory models, you must first restart the DMC instances and change the configuration before switching to a different process in the production.

5 Instantiation of a production order

The objective of the instantiation of a production order is the generation of a digital workpiece that you can transfer to the corresponding DMW for production. The instantiation is based on the production orders (requirement). The production orders include information on the variant to be produced, the manufacturing instruction and the factory model. The workpiece is a kind of data container filled with all relevant information on the production of this variant. During the production in the DMW, the system records and displays all status information referring to the production process.

During runtime, a DMC instance generates the digital workpieces. This instance is called Dynamic Workpiece Generator (DWG).

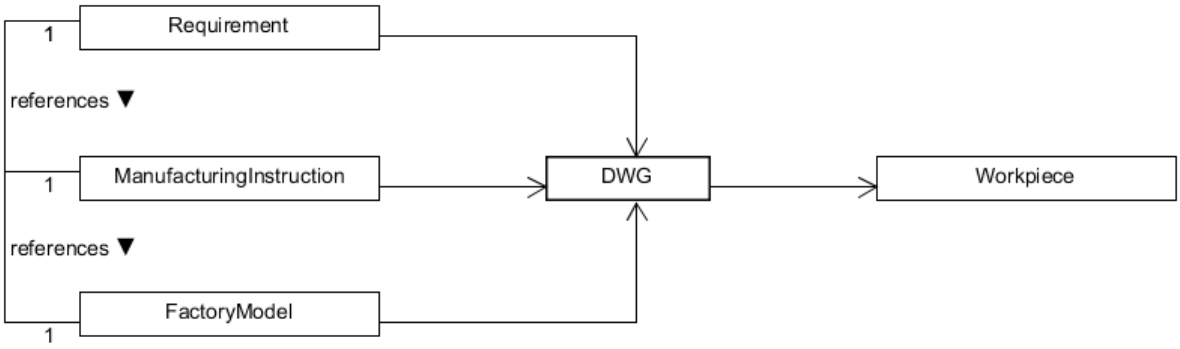

Figure 10 DWG

As shown in Figure 10 DWG, the production order (requirement) references the factory model for the line on which the order is to be produced. The order also references the manufacturing instruction describing the production of the corresponding variant. Using this information, the DWG generates a workpiece for each production order.

Data model

The workpiece is used as data container for the production process. All information on the process is stored in the workpiece and is available e.g. in case of a decision whether or not a process is interlocked. You can additionally use specific process data that is not included in the factory model and the manufacturing instruction. Additional data can be found in HYDRA or other sources. You can make an extension in the DWG to transfer more information into the workpiece than delivered by default.

The manufacturing instructions define the relevant process and work steps of the production variant. The individual steps are based on the option code conditions.

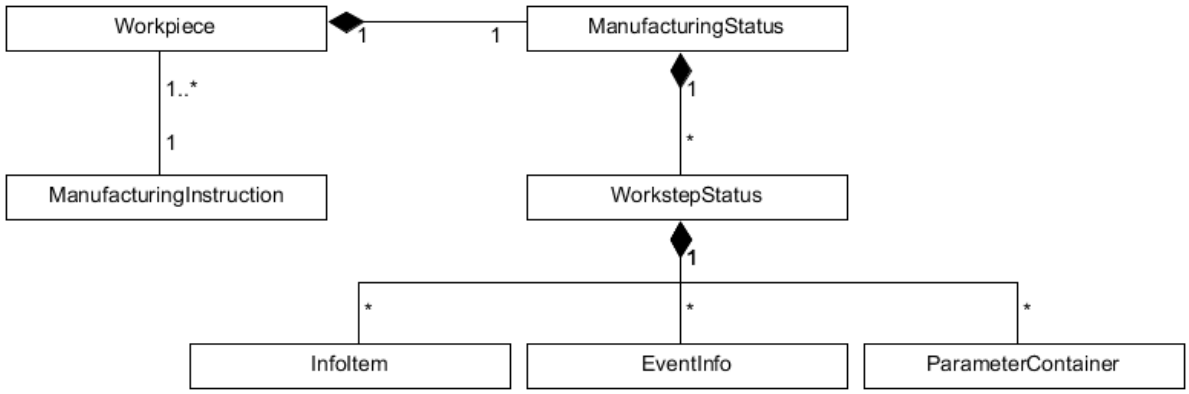

Figure 11 Workpiece

The status information (ManufacturingStatus) is structured as shown in Figure 11 Workpiece). The status container is a list of status information on the individual worksteps (WorkstepStatus). You can store different classes of information for each work step.

The InfoItem are key value pairs. The EventInfo include selected events which occurred during processing. The ParameterContainers are lists of key value pairs that were conceived for the use of individual components.

Generator Template

You can install a DMC instance in the HYDRA server to generate the workpieces. Refer to the configuration template *DWG* as basis. The configuration *config_generator.xml* shows how to use this template using the example of the reference process:

```
<CollectorConfiguration   Name="Generator"
                          Version="1.0"
                          Template="DWG"
                          FactoryModel="fm_reference.xml">

  <ComponentConfiguration Name="JtpService">
    …
  </ComponentConfiguration>

  <ComponentConfiguration Name="RequirementRepository">
    <Parameter Name="RequestInterval" Value="60" />
    <Parameter Name="InstantiatedFlagAcronym" Value="operation.userfield34" />
    <Parameter Name="OptionCodesAcronym" Value="operation.userfield66" />
    <Parameter Name="ManufacturingInstructionIdAcronym"
               Value="operation.userfield65" />
    <Parameter Name="WorkpieceIdAcronym" Value="operation.userfield60" />
    <Parameter Name="SequenceNumberAcronym" Value="operation.plan.start_ts" />
    <Parameter Name="TerminalIdAcronym" Value="operation.userfield07" />
    <Parameter Name="IsDMCOperationAcronym" Value="operation.userfield33" />
  </ComponentConfiguration>
```

```
  <ComponentConfiguration Name="ManufacturingInstructionRepository">
    <Parameter Name="ResourceDirectory" Value="instructions" />
  </ComponentConfiguration>

  <ComponentConfiguration Name="WorkpieceRepo115"
      Class="mpdv.MachineDataCollector.Dmc.Manufacturing.WorkpieceRepository">
    <Parameter Name="ResourceDirectory" Value="tnr115" />
  </ComponentConfiguration>

  <ComponentConfiguration Name="WorkpieceBuilder" Children="WorkpieceRepo115" />

</CollectorConfiguration>
```

You must first configure the connection to the HYDRA server. You can find the necessary parameters in the DMC API documentation of the component *JtpSharpService*.

Set the parameters of the component *RequirementRepository* next. Please refer to the DMC API documentation for further details on the individual parameters. Basically, you use this component to load operation data from HYDRA. With this data you then define the requirements used for the generation.

If necessary, you must change the directory where the manufacturing instructions are stored. To do so, set the parameter *ResourceDirectory* of the component *ManufacturingInstructionRepository.* A relative path is resolved relative to the shared data directory.

The generated workpieces are stored in the data system using the component *WorkpieceRepository.* The parameter *ResourceDirectory* is resolved relative to the RuntimeDataDir. In the example above, the workpieces for terminal 115 are stored in the directory c:\ProgramData\mpdv\mdc\Generator\tnr115\ using the component WorkpieceRepo115 ("WorkpieceRepo" + [TerminalId]).

If the DWG generates workpieces for several terminals, you must configure a corresponding WorkpieceRepository component for each terminal. Assign each of these components as a child to the component *WorkpieceBuilder.*

You can use the component *FileTransporter* to transport the generated workpieces to another host.

Extension of the workpiece generation

In general, you can completely adjust the workpiece generation to specific requirements. If the basic process of the generation matches the requirements, you can adjust this basic process via an extension component.

Use the interface *IWorkpieceModifier* to this end. This interface defines the method

```
void ModifyWorkpiece(Workpiece workpiece, Requirement requirement);
```

If you implement this interface, you can adjust the generated workpiece by e.g. transferring additional information into the container. In addition to the workpiece, the requirement is transferred as parameter to this method. You can now implement the interface according to the requirements.

In the reference process, an example could be the component *VinGenerator* (VinGenerator.cs), which generates the Workpiece ID as VIN (Vehicle Identification Number). In order to include the component, add it to the configuration and assign it as child to the WorkpieceBuilder component.

Such an extension could be applied if you want to provide the components in use with additional information during runtime. To this end, the ManufacturingStatus offers the ParameterContainer. You can add these containers to the workpiece using the method *AddParameterContainer().*

Further information on the helpers used to process workpieces is included in the DMC API documentation (class *WorkpieceHelper*).

6 Deployment

The following section discusses different deployment aspects. The reference process is used as an example to develop a deployment strategy. In addition to the aspects mentioned in this section, the DMC hardware and software requirements still apply.

The main components to connect a line include the following functions:

- **HYDRA**: Provides order information and reports
- **DWG** (Dynamic Workpiece Generator): Creates workpieces and is based on the order information
- **DMW** (Dynamic MES Weaver): Connects the line and performs the production process of the workpieces
- **DLP** (Dynamic Line Panel): Terminals that display and control the production process

We suggest the configuration shown in Figure 12 Deployment) to deploy the above mentioned components.

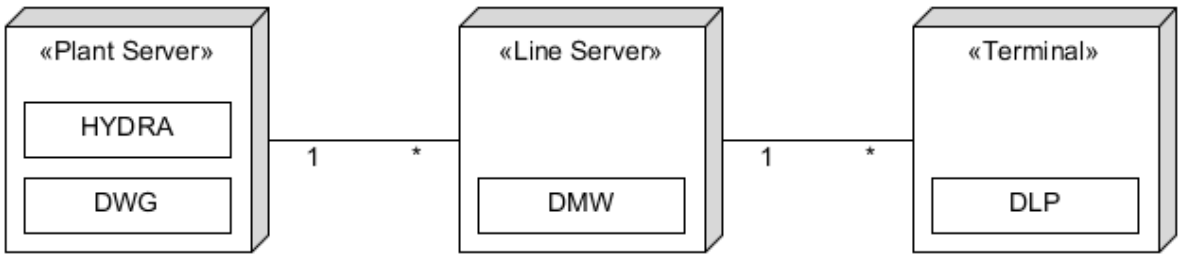

Figure 12 Deployment

The DWG is installed on the HYDRA server. If this is not possible, you can install the DWG on the line server, together with the DMW. The DMW of a line should run on a dedicated system (virtualization is possible). Install the DLPs in a custom system. The DLPs connect to the DMW in the line server. Failure safety is especially important with the line server. In general, failure-safety should be guaranteed for all servers and terminals and the network connections.

Following this strategy, the reference process is deployed as shown in Figure 13 Deployment of the reference line). DWG and DMW are installed in different servers and each DLP is installed in a dedicated terminal. The example of this deployment also shows how the DMW driver connection can be transferred to another process via the configuration config_drivers.xml. The general structure of the configuration config_drivers.xml corresponds to the structure of a DLP. The driver component is registered in the DMW and communication with the DMW is provided by inter-process communication.

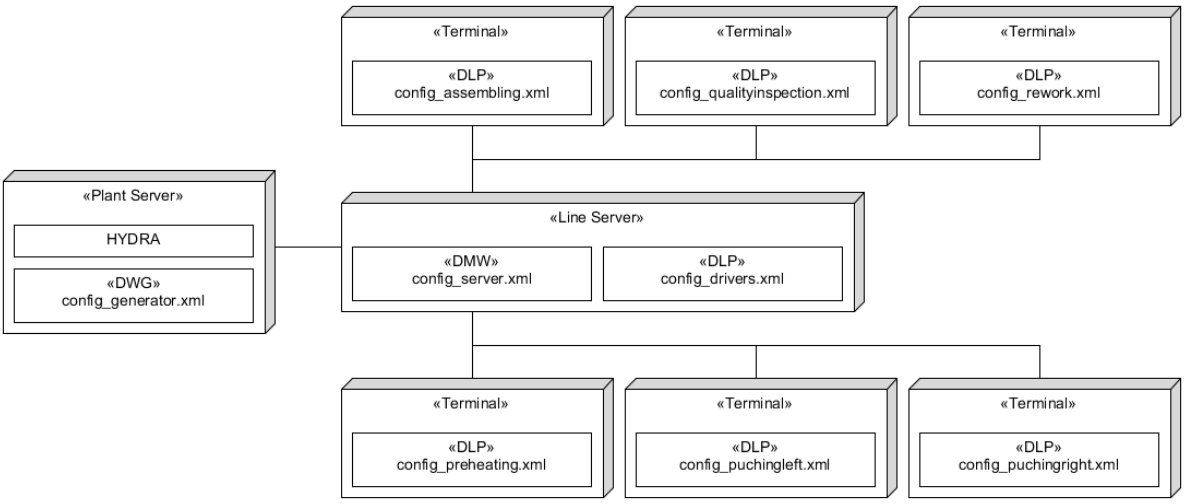

Figure 13 Deployment of the reference line